

Design by Contract

Bertrand Meyer, no seu trabalho sobre "Design by Contract", propõe uma visão onde a confiabilidade do software, composta pela correção e pela robustez, é alcançada através de uma metáfora de decisões de contratação documentadas. O centro desta filosofia é a ideia de que a relação entre uma rotina e quem a chama deve ser regida por um contrato rigoroso, onde as obrigações e benefícios de ambos os lados são explicitados por meio de asserções. Estas asserções dividem-se em precondições, que são exigências que o cliente deve satisfazer para que a chamada seja correta, e pós-condições, que garantem as propriedades asseguradas pelo fornecedor após a execução bem-sucedida. Complementando estes conceitos, existem os invariantes de classe, que definem propriedades que todos os objetos de uma determinada classe devem satisfazer em todos os estados "observáveis", transcendendo as rotinas individuais.

A aplicação consistente deste modelo permite a rejeição da chamada "programação defensiva", na qual cada módulo tenta proteger-se com checagens redundantes. Meyer argumenta que, se as responsabilidades forem claramente atribuídas através de contratos, a redundância desnecessária é eliminada, resultando em arquiteturas mais simples e, conseqüentemente, mais confiáveis. No caso de falhas, o autor define que uma exceção surge justamente quando uma das partes não consegue cumprir o seu contrato. Para lidar com isso, ele propõe um mecanismo disciplinado de tratamento de exceções baseado em duas respostas possíveis: a retomada (resumption), onde se tenta corrigir o problema e repetir a operação via retry, ou o pânico organizado (organized panic), onde a rotina limpa o estado do objeto para restaurar o invariante e reporta a falha ao chamador através de uma cláusula rescue. A analogia do cozinheiro e do corpo de bombeiros ilustra bem este papel: enquanto o cozinheiro deve servir a refeição e manter o restaurante seguro, os bombeiros apenas precisam garantir que o restaurante retorne a um estado estável, sem a obrigação adicional de servir o jantar.

Além disso, o paradigma contratual oferece uma nova perspectiva sobre a herança e a vinculação dinâmica, interpretando a redefinição de rotinas como uma forma de "subcontratação". Seguindo a lógica da honestidade contratual, um subcontratado (classe descendente) pode realizar o trabalho de forma melhor, mas nunca de forma mais exigente ou entregando menos do que o prometido originalmente. Tecnicamente, isto traduz-se na regra de que a pré-condição de uma rotina redefinida deve ser mais fraca ou igual à original, enquanto a pós-condição deve ser mais forte ou igual. Meyer conclui o seu argumento defendendo o uso de funções parciais, aceitando que as rotinas podem não funcionar para todos os casos de entrada possíveis, desde que os seus limites e domínios sejam claramente estabelecidos por contratos. Esta abordagem foca na coerência das interfaces e na simplicidade dos módulos como o caminho real para a construção de sistemas robustos e realistas.